

Generator (g):

[Another Generator](#)

[Chooses a primitive root modulo p](#)

Important: Both Alice and Bob can see the same public values (p , g). These are not secret and can be transmitted over an insecure channel.

Step 2: Private Key Generation and Public Key Calculation

Each participant generates their own private key (secret) and calculates their public key using the formula: **Public Key** = $g^{(\text{private key})} \bmod p$

Alice

Private Key (a):

[Generate A](#)

Alice's secret private key (kept secret)

Public Key ($g^a \bmod p$):

[Calculate \$g^a\$](#)

Alice's public key (can be shared openly)

[Send Public Key to Bob](#)

Received from Bob:

Bob's public key ($g^b \bmod p$)

Shared Secret:

[Calculate Shared Secret](#)

Final shared secret: $(g^b)^a \bmod p = g^{(ab)} \bmod p$

Bob

Private Key (b):

[Generate B](#)

Bob's secret private key (kept secret)

Public Key ($g^b \bmod p$):

[Calculate \$g^b\$](#)

Bob's public key (can be shared openly)

[Send Public Key to Alice](#)

Received from Alice:

Alice's public key ($g^a \bmod p$)

Shared Secret:

[Calculate Shared Secret](#)

Final shared secret: $(g^a)^b \bmod p = g^{(ab)} \bmod p$

Step 3: Verification

Success! If both Alice and Bob calculated the same shared secret value, they have successfully established a secure communication channel. This shared secret can now be used as an encryption key for further communication.

Security Note: Even though an eavesdropper can see all the public information (p , g , $g^a \bmod p$, $g^b \bmod p$), they cannot easily compute the shared secret $g^{(ab)} \bmod p$ without solving the discrete logarithm problem, which is computationally difficult for large numbers.